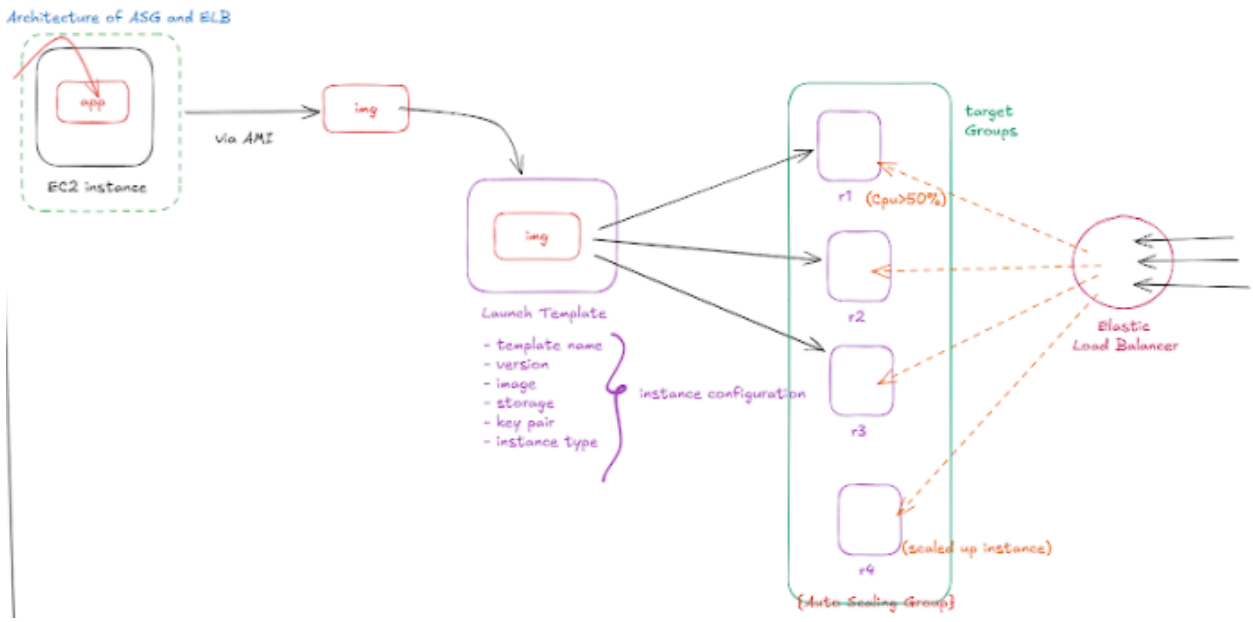
End-to-End Static Website Deployment on AWS with ASG and Load Balancer

Here we are deploying the static web application on AWS using the Auto Scaling Groups to achieve high availability and scalability.
AWS Services used :

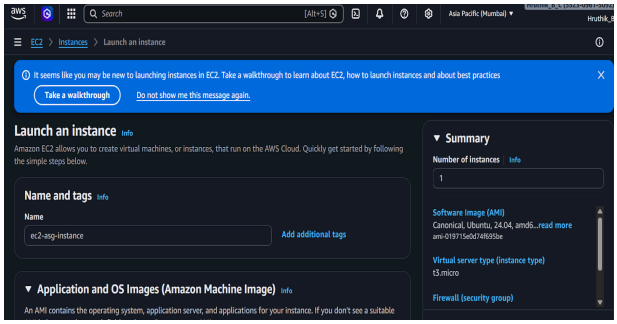
- EC2 Instance.
- Amazon Machine Image(AMI).
- Launch Template.
- Security Groups.
- CloudWatch.
- Application Load Balancer(ALB).

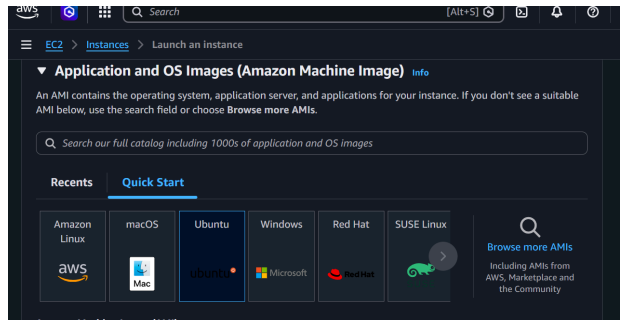
Infrastructure Setup

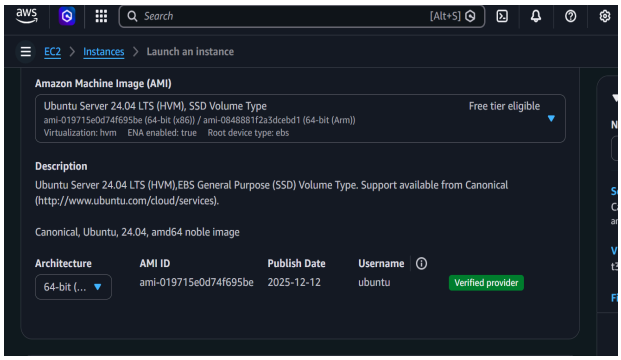


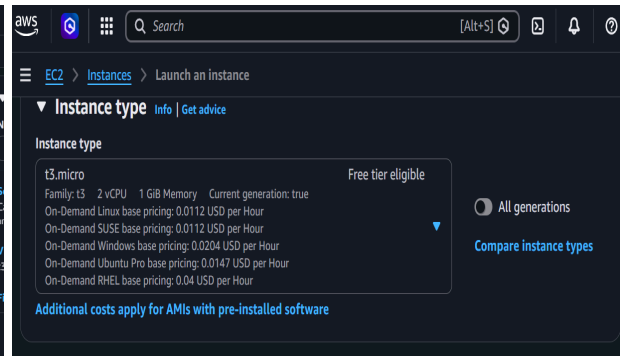
AMI - It is a template which contains the software configurations like OS, app server etc
EC2 instance - This is the first step in which we are requesting the aws for the server where we can host our static web application. Below are the steps to fill the form or the data to request the server :

- Under EC2 dashboard service in AWS console click on the instance component and then click on the launch instance and provide the instance name.
- In my project we are using the on-demand instances which offer stable, uninterrupted compute at standard pricing, while spot instances provide significant cost saving at the risk of interruption but my application need not be interrupted during the application in the production environment.
- Whenever there is a priority requirement and we need those dedicated servers we use on-demand instances.

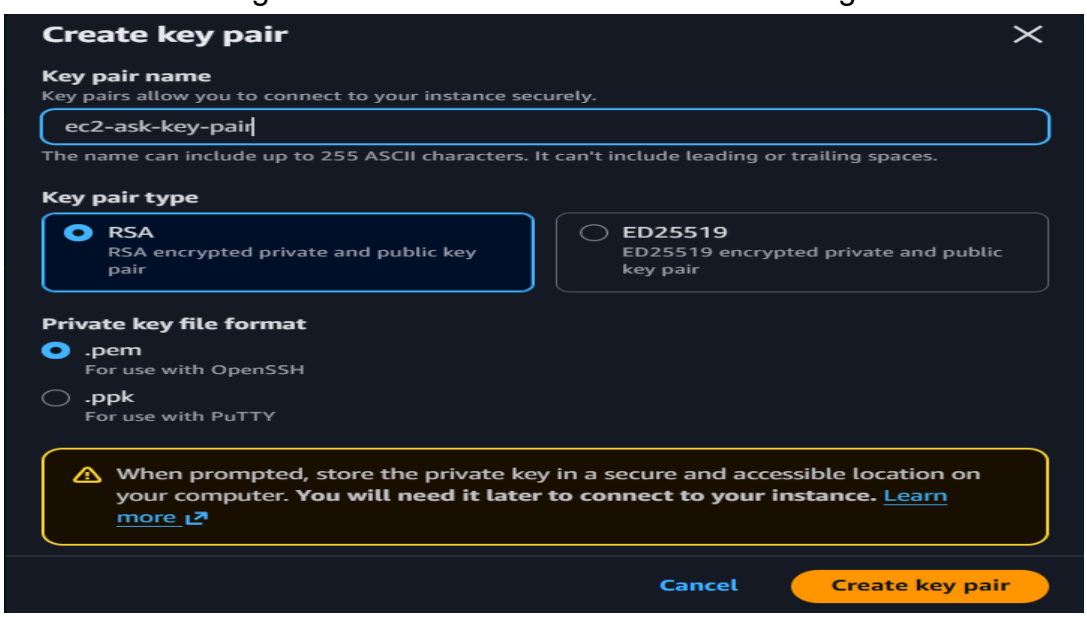






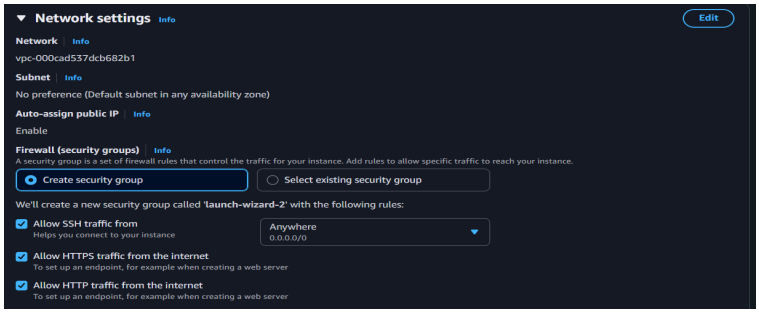


Once after providing the instance name under the AMI section choose any of the distribution as a new learner I am using the free tier AMI with 64 bit.
Then under Instance type choose any of the instance type based on your requirement
As of now we will go with the t3.micro which has free tier eligible.



After selecting the instance type then click create a new key-pair. Basically using Key-pair we can securely connect to the created EC2 instance and there are two types of Key-pairs i. Public key-pair : Through this Key-pair everyone can access the created EC2 instance.

ii. Private key-pair : Only the user who has created the EC2 instance will be able to access the instance.

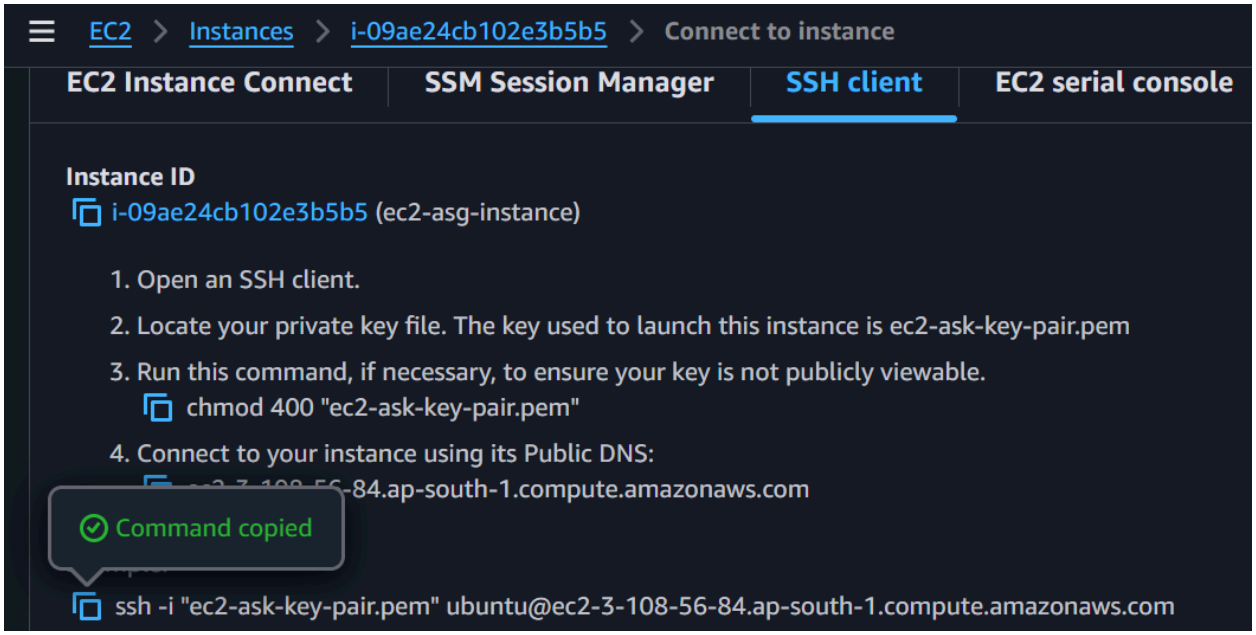


Once the key-pair is generated and attached to the EC2 instance under select the create a security group then allow the SSH, HTTP, HTTPS traffic from the internet.
At last verify all the details carefully and then click on launch the instance.

There are several steps if need to connect to the EC2 instance :
If we need to connect to the created EC2 via SSH client the port no is 22 and if we attach any Elastic Block Stores volume till 30GB there will be free

So basically to run the static web application we need Nginx or Tomcat or HTTPD server in the EC2 instance. Here we are using the Nginx server. Steps to install the nginx server in the EC2 instance.

First we need to connect to EC2 instance
There are two ways to connect the EC2 instance
i. EC2 instance connect.
ii. SSH client.
To connect EC2 instance via SSH client :



As we can see in the image, copy the command to the clipboard and paste over the command prompt in the windows machine or terminal in the Mac OS.

```
ubuntu@ip-172-31-41-112: ~

PS C:\Users\BIT\Downloads> ssh -i "ec2-ask-key-pair.pem" ubuntu@ec2-3-108-56-84.ap-south-1.compute.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Feb 25 01:33:36 UTC 2026

  System load:  0.03               Temperature:           -273.1 C
  Usage of /:   26.4% of 6.71GB    Processes:             116
  Memory usage: 25%                Users logged in:       0
  Swap usage:   0%                 IPv4 address for ens5: 172.31.41.112


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Wed Feb 25 01:31:40 2026 from 157.50.206.200
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-41-112:~$
```

After performing the execution we should be able to see this interface as expected.

Once the EC2 instance is connected perform these commands to install the Nginx server on the EC2 instance.

sudo apt update : This command used to update the package manager in the ubuntu distribution.

sudo apt-get install nginx -y : This command is used to install the nginx server into our EC2 instance.

nginx –version : This command is used to check whether the Nginx server installed properly or not into our instance.

After installing the Nginx server perform the below commands that are related to Nginx server :

sudo systemctl start nginx : By using this command we can start the Nginx server

sudo systemctl status nginx : By this command we are verifying whether the nginx is up or not basically running or not.

sudo systemctl enable nginx : By this command we are enabling all the processes in the Nginx server.

sudo systemctl stop nginx : By this command we are stopping the Nginx server.

Once the Nginx server setup is done properly the host the static web application which is present in the EC2 instance.

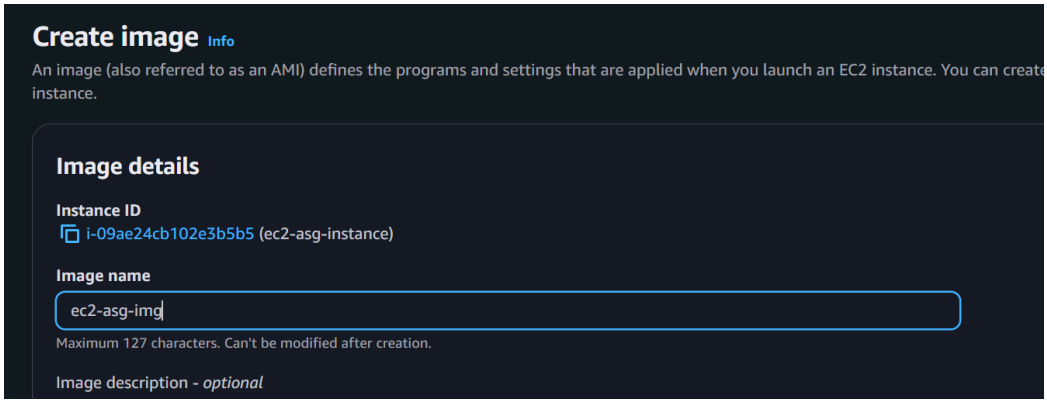
The process to host the application in Nginx server :

```
ubuntu@ip-172-31-41-112:~$ sudo cp -r 2153_fireworks_composer/* /var/www/html
ubuntu@ip-172-31-41-112:~$ cat /var/www/html/
cat: /var/www/html/: Is a directory
ubuntu@ip-172-31-41-112:~$ cat /var/www/html/index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Fireworks Composer - New Year 2026</title>
```

Copy the Folder to the Nginx path (/var/www/html)
Then verify that the application is working with the public ip address of the EC2 instance and the default port number of the Nginx server and HTTP request is 80.

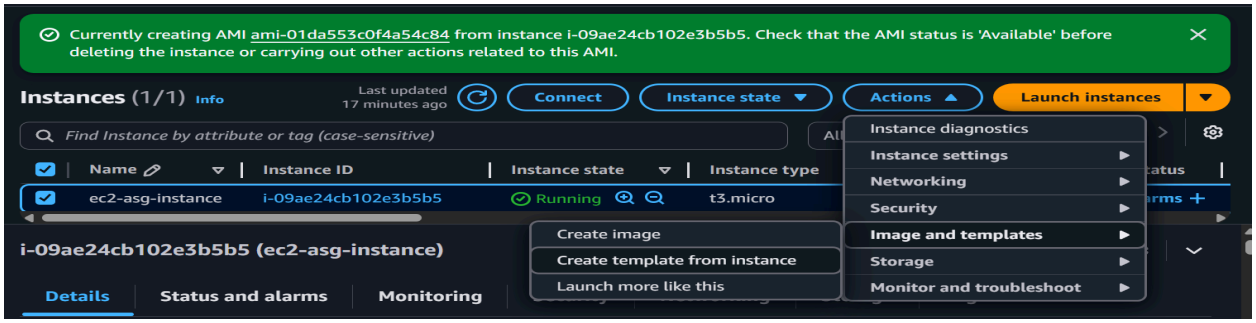
Make sure you have accessed the application successfully.

Once the web application is hosted on the EC2 instance make image out of the EC2 instance :

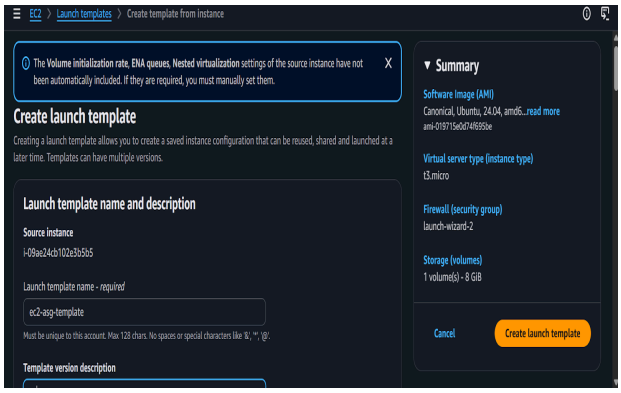


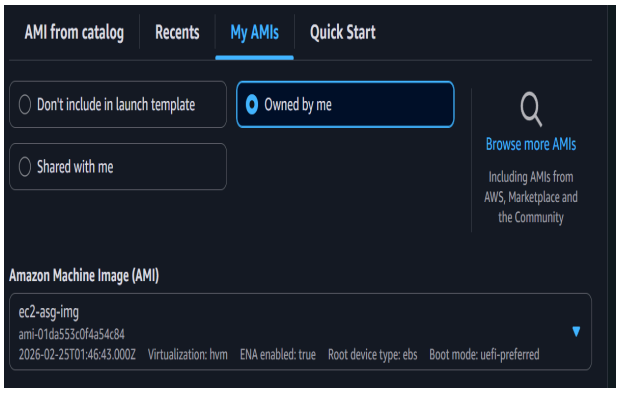
Provide the name and just create the image, Once the image is created then verify the status of the image in the AMI dashboard It should be in Available state.

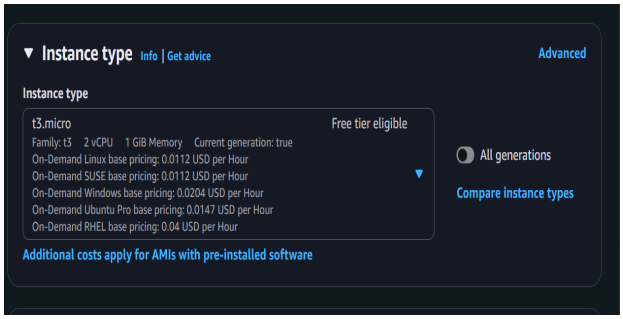
After creation of the image with using the same image we will make a template out of it:

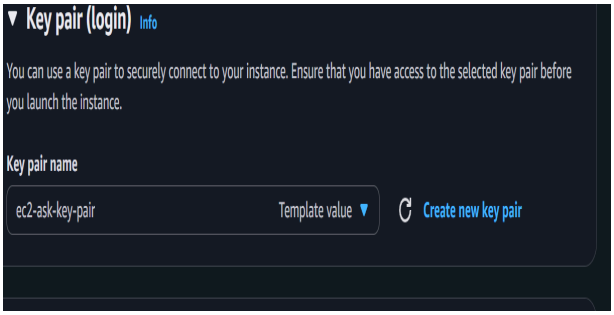


As you can see in the above image click on create template from instance then give the name for the template









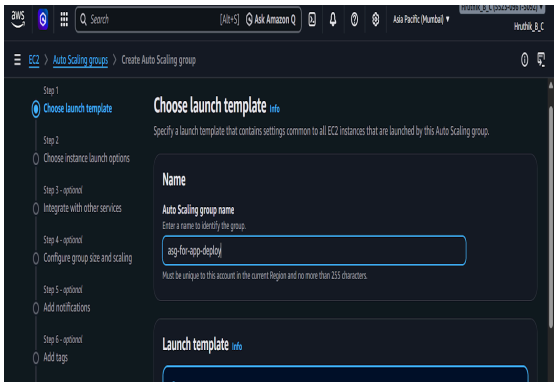
A Launch Template named `ec2-asg-template` was created from a pre-configured EC2 instance. It includes a custom AMI, instance type (t3.micro), security group, key pair, and EBS storage configuration. The template standardizes instance provisioning and is used by the Auto Scaling Group to automatically launch consistent, On-Demand EC2 instances across multiple Availability Zones.

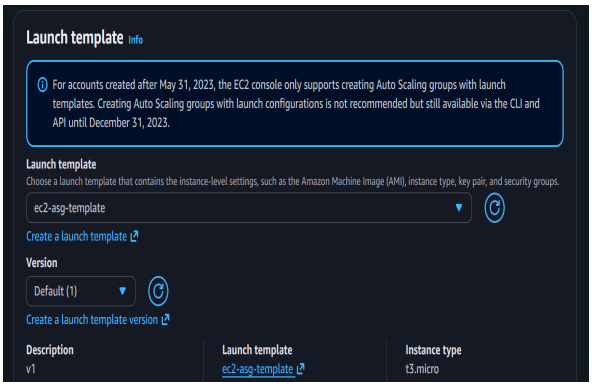
Auto Scaling Group (ASG) Configuration Using Launch Template

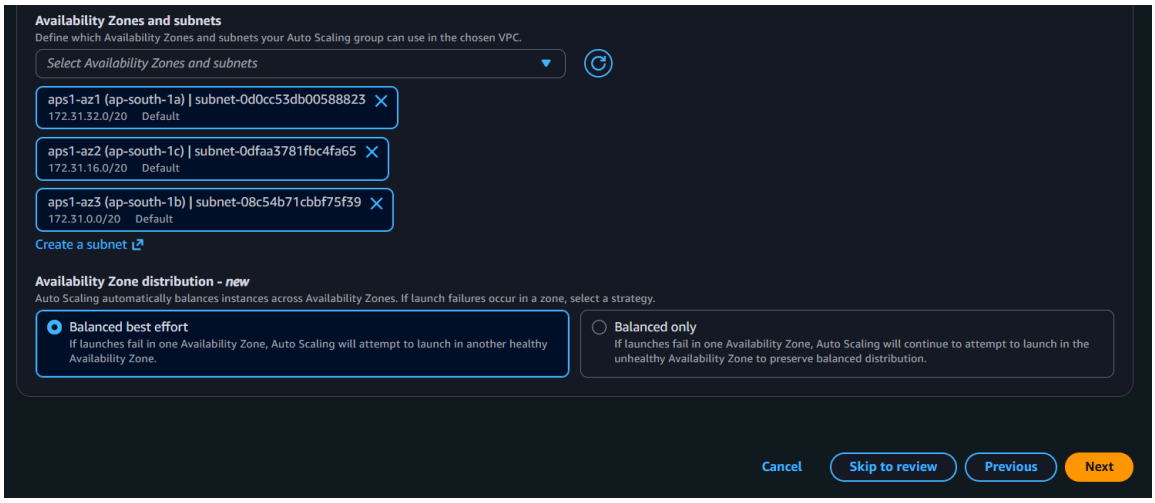
Once the Launch Template is created, it is attached to an Auto Scaling Group (ASG) to enable automatic scaling, and lifecycle management of EC2 instances.

ASG configuration steps :

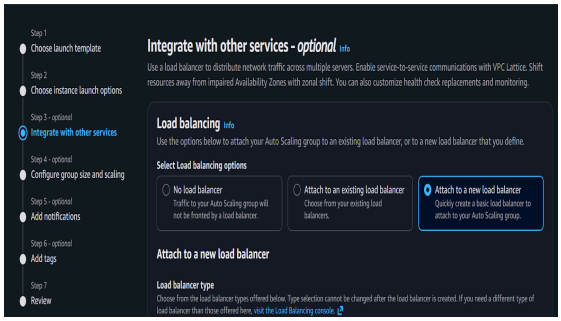
- Navigate to **EC2 → Auto Scaling Groups → Create Auto Scaling Group.**
- Provide a unique ASG name.
- Then attach the launch template which earlier we have created.
- Then define the availability zones at least 2 AZs.
- Select the Balanced best effort when the instance in one AZ is down still other instances which are in the other AZs will be working fine.

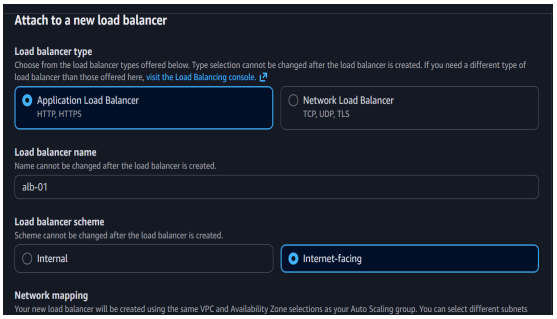


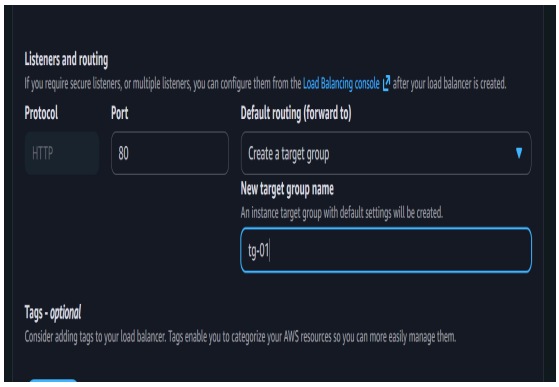


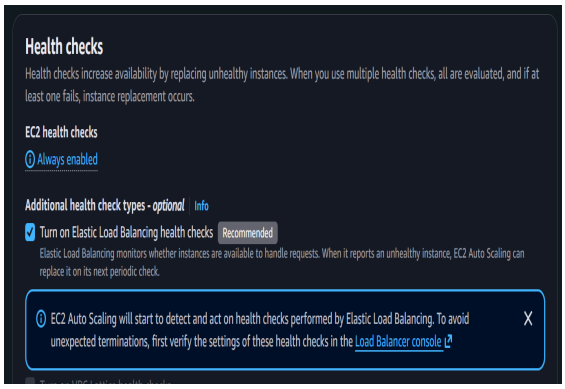


Once completing this process click on the next button.

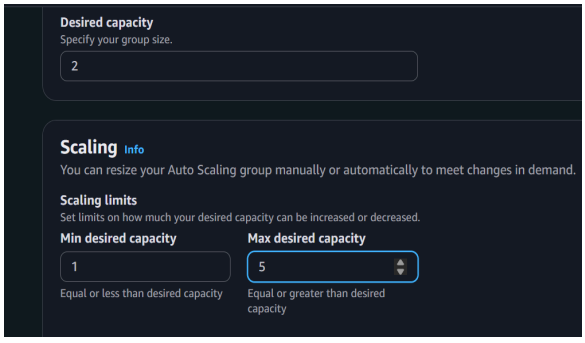


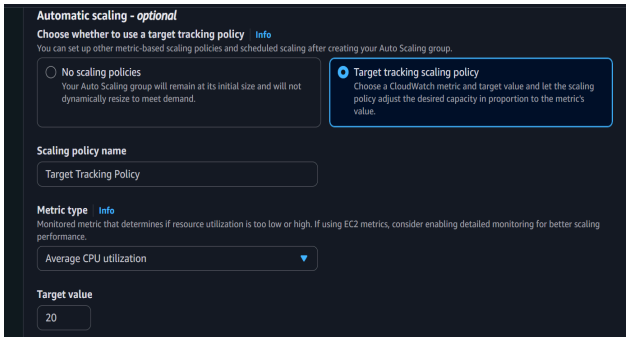






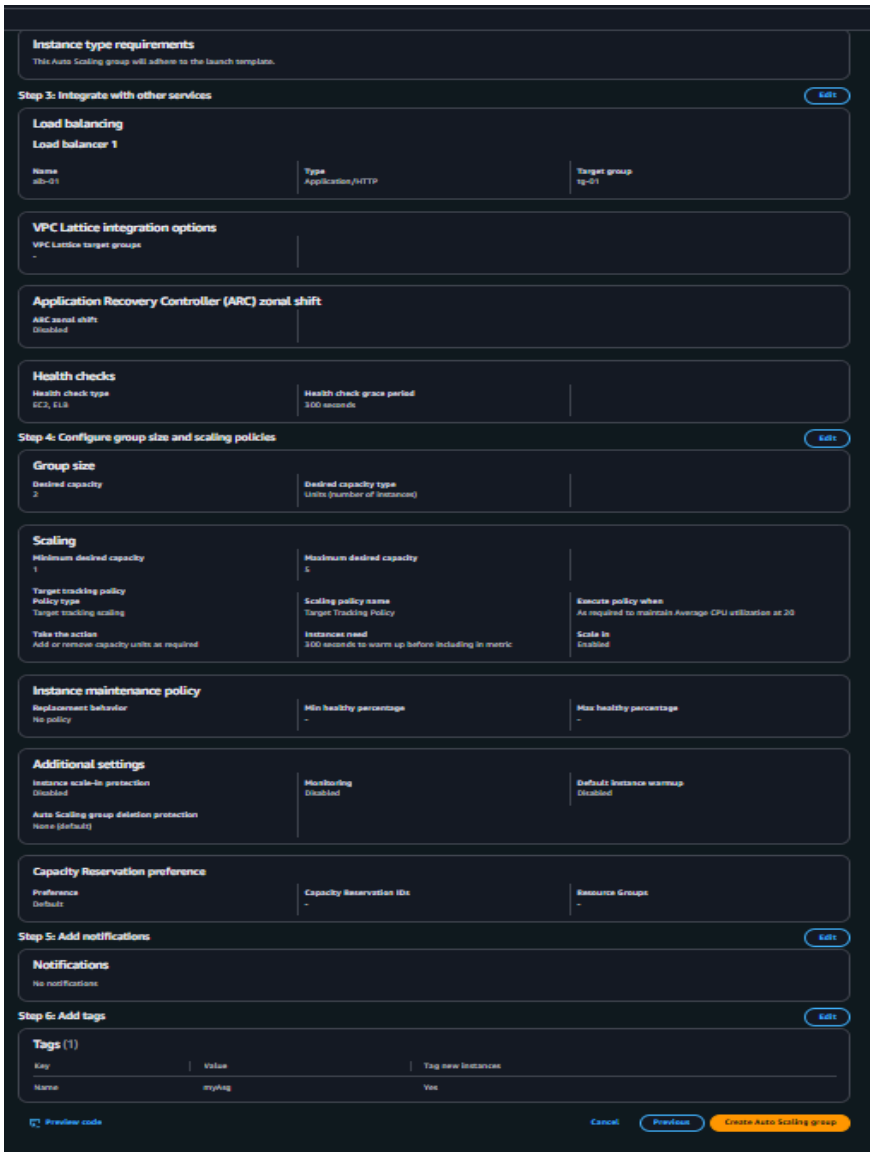
The Auto Scaling Group was integrated with a newly created Internet-facing Application Load Balancer. The load balancer listens on HTTP port 80 and forwards traffic to an instance-based target group. Elastic Load Balancing health checks were enabled, allowing the Auto Scaling Group to automatically replace unhealthy instances and maintain high availability of the static web application.



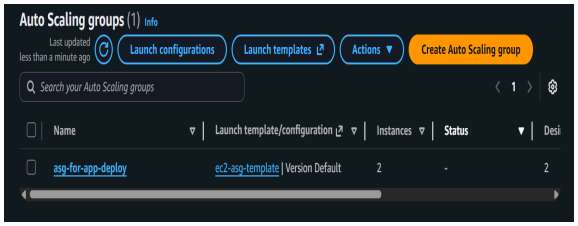


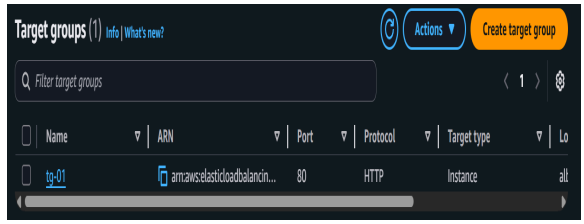
The Auto Scaling Group was configured with a desired capacity of two instances, a minimum capacity of one instance, and a maximum capacity of five instances. A target tracking scaling policy based on average CPU utilization was applied with a target value of 20%. This configuration allows the infrastructure to automatically scale in or out based on workload demand while maintaining high availability and cost efficiency.

After reviewing all configurations, the Auto Scaling Group was finalized with a launch template–based instance configuration, integrated Application Load Balancer, multi-AZ deployment, target tracking scaling policy, and defined capacity limits. This setup ensures high availability, automatic scaling, and self-healing behavior for the static web application.

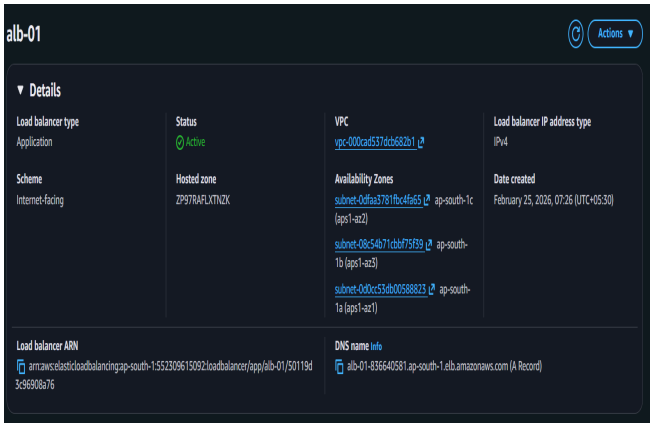


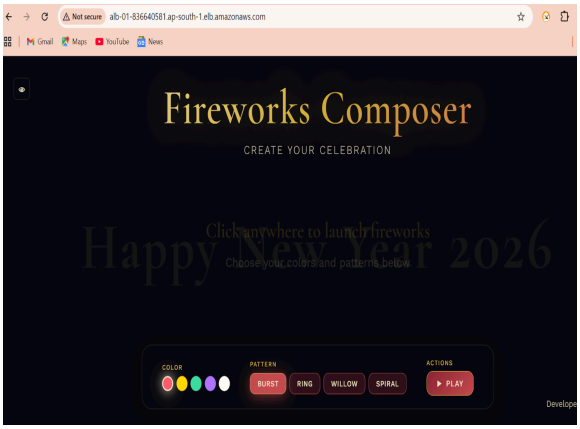
Once after clicking on the create auto scaling group verify the Target Groups, Auto Scaling Groups, Load balancers and the two newly desired instances are created or not in the EC2 dashboard and verify with the names given during the creation of those.





Once all the verification of the resources are done and validating it successfully perform the below steps :





The Application Load Balancer `alb-01` was verified to be active, Internet-facing, and deployed across multiple Availability Zones. The load balancer DNS name was accessed successfully from a browser, confirming public accessibility of the static web application. Security groups were properly configured to allow HTTP traffic, ensuring secure and reliable request routing to EC2 instances managed by the Auto Scaling Group.

Then in the instance dashboard connect to any one of the replicas we are created by the auto scaling groups and then manually add the load to instances because as of now we don't have the users for our application so that by performing the below command in the instance we can add the load to our instance

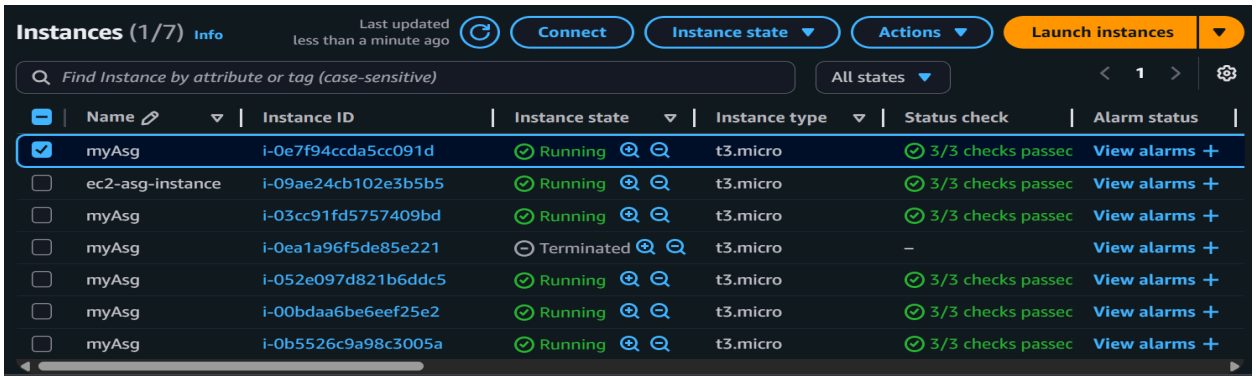
sudo apt-get install stress : This command is used to install the stress package.

After installing the stress package use the below command to increase the load to the instance.

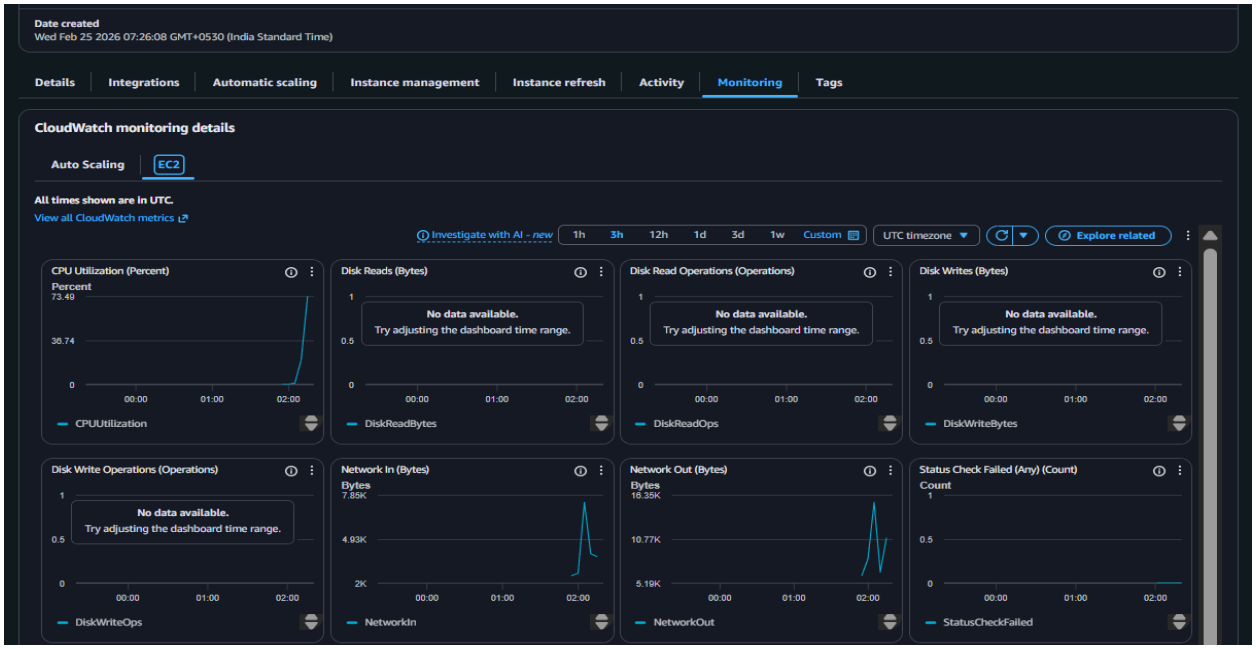
```
stress --cpu 2 --timeout  300
```

```
root@ip-172-31-34-236:~# stress --cpu 3 --timeout 300
stress: info: [2890] dispatching hogs: 3 cpu, 0 io, 0 vm, 0 hdd
```

Here we validated Auto Scaling by running a CPU stress test for five minutes, which increased CPU utilization and triggered a scale-out event based on the target tracking policy.



After performing the CPU stress test, Amazon CloudWatch metrics were monitored to validate CPU utilization behavior and Auto Scaling Group response and as the load increases the EC2 instances gets scaled up according to the load.



CloudWatch monitoring confirmed the impact of the CPU stress test by showing a significant spike in CPU utilization, reaching approximately 73%. This exceeded the target tracking threshold and triggered the Auto Scaling Group to scale out automatically. Network metrics also reflected active traffic handling, while all instances remained healthy. This validated the effectiveness of CloudWatch metrics, scaling policies, and the self-healing behavior of the deployed infrastructure. Once the traffic gets reduced the instances will become the desired number of instances.

Conclusion :

This project successfully demonstrates the end-to-end deployment of a scalable and highly available static web application on AWS using industry-standard cloud architecture practices. By leveraging EC2, AMI, Launch Templates, Auto Scaling Groups, Application Load Balancer, and CloudWatch, the infrastructure was designed to automatically adapt to changing workloads while maintaining reliability and performance.

The use of target tracking Auto Scaling policies ensured that compute resources scaled dynamically based on real-time CPU utilization. This behavior was practically validated using a controlled CPU stress test, which triggered automatic scale-out events and was clearly reflected in CloudWatch metrics. Once the load reduced, the system stabilized back to the desired capacity, demonstrating cost efficiency and elasticity.

Overall, this implementation validates key cloud concepts such as high availability, fault tolerance, self-healing, and observability, making it a strong example of a production-ready AWS architecture and a solid foundation for real-world DevOps and cloud engineering scenarios.